DATABASE NORMALIZATION

1. What is Normalization?

Normalization is the process of organizing a relational database to reduce data redundancy and improve data integrity. It was introduced by Edgar F. Codd (the inventor of the relational model) in 1970. The process involves decomposing large, poorly structured tables into smaller, well-structured tables while preserving the relationships between data.

> 🕮 **Definition**
> Normalization = a step-by-step process of converting a relation (table) into a standard form by applying a set of rules called Normal Forms (NF), ensuring each piece of information is stored only once.

2. Why Do We Need Normalization?

Before understanding Normal Forms, it is essential to understand the problems that arise in an un-normalized (raw) database. These problems are called Anomalies.

2.1 The Three Data Anomalies

Consider the following un-normalized table storing student course and faculty data:

Std ID	Std Name	Course ID	Course Name	Faculty Name	Faculty Phone
101	Alice	CS101	DBMS	Dr. Sharma	9876543210
101	Alice	CS102	OS	Dr. Mehta	9876500001
102	Bob	CS101	DBMS	Dr. Sharma	9876543210
103	Carol	CS103	Networks	Dr. Rao	9876522222

Anomaly Type	What Happens?	Example (from above table)
Insertion Anomaly	Cannot insert data without having other unrelated data.	Cannot add a new faculty unless they teach a course with an enrolled student.
Update Anomaly	Updating one fact requires changing many rows.	If Dr. Sharma changes phone number, must update ALL rows where she appears.
Deletion Anomaly	Deleting a row accidentally removes other useful data.	If Carol drops Networks, we lose all info about Dr. Rao.

3. Key Concepts Before Normal Forms

3.1 Functional Dependency (FD)

A Functional Dependency X → Y means: for every value of X, there is exactly one value of Y. We say X determines Y, or Y is functionally dependent on X.

FD	Meaning	Example
Std_ID → Std_Name	Each student ID has one name	101 → Alice
Course_ID → Course_Name	Each course ID has one name	CS101 → DBMS
Faculty_Name → Faculty_Phone	Each faculty has one phone	Dr. Sharma → 9876543210

3.2 Types of Keys

Key Type	Definition	Example
Super Key	Any set of attributes that uniquely identifies a row	{Std_ID}, {Std_ID, Name}
Candidate Key	Minimal super key — no unnecessary attributes	{Std_ID}
Primary Key	Chosen candidate key to uniquely identify rows	Std_ID chosen as PK
Foreign Key	Attribute referencing the primary key of another table	Course_ID in Enrollment references Course
Composite Key	Primary key made of two or more attributes	{Std_ID, Course_ID}
Prime Attribute	Attribute that is part of any candidate key	Std_ID, Course_ID
Non-Prime Attribute	Attribute NOT part of any candidate key	Std_Name, Faculty_Name

4. Normal Forms — Visual Overview

Each Normal Form builds on the previous one. Think of it as a staircase:

BCNF	Every determinant is a candidate key (strongest standard form)
3NF	In 2NF + no transitive dependency of non-prime attributes on PK
2NF	In 1NF + no partial dependency (non-prime attr fully depends on whole PK)
1NF	Atomic values only, no repeating groups, each row uniquely identifiable
UNF	Un-Normalized Form — raw data with repeating groups and redundancy

5. First Normal Form (1NF)

First Normal Form (1NF)	
Rule	A table is in 1NF if: (1) All values are atomic (indivisible). (2) Each column has a single value per row. (3) Each row is unique (primary key exists). (4) No repeating groups or arrays in cells.
✓ Pros	• Eliminates multi-valued and repeating attributes • Makes the table queryable using standard SQL • Foundation for all higher normal forms
✗ Cons	• Still contains data redundancy (if composite PK, partial deps exist) • Anomalies not fully resolved
★ Keys	★ Every cell must have ONE and ONLY ONE value ★ You must define a primary key ★ Repeating groups must be moved to separate rows ★ Multi-valued attributes (e.g., 'phone1, phone2') must be split

5.1 Before 1NF (Violates 1NF)

Std_ID	Name	Courses (VIOLATION!)	Phone Numbers
101	Alice	DBMS, OS, Networks	9876543210, 9876500001
102	Bob	DBMS	9988776655

↓ CONVERT TO 1NF: Split multi-valued attributes into separate rows

5.2 After 1NF

Std_ID	Name	Course	Phone	(PK composite)
101	Alice	DBMS	9876543210	← *Std_ID + Course*
101	Alice	OS	9876543210	
101	Alice	Networks	9876543210	
102	Bob	DBMS	9988776655	

6. Second Normal Form (2NF)

2NF applies only when the table has a COMPOSITE primary key.

Second Normal Form (2NF)	
Rule	A table is in 2NF if: (1) It is already in 1NF. (2) Every non-prime attribute is fully functionally dependent on the ENTIRE primary key (no partial dependency).
✓ Pros	• Removes partial dependencies — reduces redundancy

	• Eliminates update and insertion anomalies related to partial deps • Produces more focused, meaningful tables
✗ Cons	• Transitive dependencies may still exist • Still may have some anomalies
★ Keys	★ Partial dependency: non-prime attr depends on PART of PK, not whole PK ★ Only relevant when PK is composite (2+ columns) ★ Solution: move partially dependent attributes to a new table ★ If PK has only one column, 1NF automatically satisfies 2NF

6.1 Partial Dependency Identified

Table: Enrollment(Std_ID, Course_ID, Std_Name, Course_Name, Marks)

Primary Key: {Std_ID, Course_ID} (composite)

Std_ID (PK)	Course_ID (PK)	Std_Name	Course_Name	Marks
101	CS101	Alice	DBMS	85
101	CS102	Alice	OS	78
102	CS101	Bob	DBMS	90

Partial Dependencies found:

- Std_ID → Std_Name (Std_Name depends only on Std_ID, not the full PK)
- Course_ID → Course_Name (Course_Name depends only on Course_ID, not the full PK)
- {Std_ID, Course_ID} → Marks ✓ (Marks correctly depends on full composite PK)

↓ **DECOMPOSE TO 2NF: Move partial dependencies to new tables**

6.2 After 2NF — Three Tables

Table 1: Student(Std_ID, Std_Name)

Std ID (PK)	Std Name
101	Alice
102	Bob

Table 2: Course(Course_ID, Course_Name)

Course_ID (PK)	Course_Name
CS101	DBMS
CS102	OS

Table 3: Enrollment(Std_ID, Course_ID, Marks)

Std_ID (FK)	Course_ID (FK)	Marks
101	CS101	85
101	CS102	78
102	CS101	90

7. Third Normal Form (3NF)

Third Normal Form (3NF)	
Rule	A table is in 3NF if: (1) It is already in 2NF. (2) There is no transitive dependency — no non-prime attribute depends on another non-prime attribute.
✓ Pros	• Removes transitive dependencies — further reduces redundancy • Prevents most update and deletion anomalies • Most real-world databases aim for 3NF • Lossless join and dependency preserving decomposition possible
✗ Cons	• Tables may increase in number • Queries may require more JOINs • BCNF anomalies may still exist in rare cases
★ Keys	★ Transitive dep: A → B → C means A → C is transitive (C depends on A via B) ★ Non-prime attribute must NOT determine another non-prime attribute ★ Emp_ID → Dept_ID → Dept_Name: Dept_Name transitively depends on Emp_ID ★ Solution: separate Dept info into its own table

7.1 Transitive Dependency Identified

Table: Employee(Emp_ID, Emp_Name, Dept_ID, Dept_Name, Dept_Head)

Primary Key: Emp_ID

Emp_ID (PK)	Emp_Name	Dept_ID	Dept_Name	Dept_Head
E01	Alice	D01	Engineering	Mr. Kumar
E02	Bob	D01	Engineering	Mr. Kumar
E03	Carol	D02	Marketing	Ms. Priya

Transitive Dependencies found:

- Emp_ID → Dept_ID ✓ (direct — fine)
- Dept_ID → Dept_Name (Dept_Name depends on Dept_ID, not Emp_ID)
- Dept_ID → Dept_Head (Dept_Head depends on Dept_ID, not Emp_ID)
- **So Emp_ID → Dept_Name is TRANSITIVE — violates 3NF!**

↓ DECOMPOSE TO 3NF

7.2 After 3NF — Two Tables

Table 1: Employee(Emp_ID, Emp_Name, Dept_ID)

Emp_ID (PK)	Emp_Name	Dept_ID (FK)
E01	Alice	D01
E02	Bob	D01
E03	Carol	D02

Table 2: Department(Dept_ID, Dept_Name, Dept_Head)

Dept_ID (PK)	Dept_Name	Dept_Head
D01	Engineering	Mr. Kumar
D02	Marketing	Ms. Priya

8. Boyce-Codd Normal Form (BCNF)

BCNF is a stronger version of 3NF, proposed by R.F. Boyce and E.F. Codd in 1974. It addresses rare anomalies that 3NF misses when there are multiple overlapping candidate keys.

Boyce-Codd Normal Form (BCNF)	
Rule	A table is in BCNF if: (1) It is already in 3NF. (2) For every functional dependency X → Y, X must be a candidate key (or super key). In other words: the left-hand side of every non-trivial FD must be a candidate key.
✓ Pros	• Eliminates all anomalies caused by FDs (stronger than 3NF) • Guarantees no redundancy from functional dependencies • Preferred form for most well-designed schemas
✗ Cons	• May NOT always preserve all functional dependencies • Decomposition may lose the ability to enforce some constraints • More aggressive splitting may complicate queries
★ Keys	★ 3NF allows non-prime attrs to depend on candidate keys; BCNF does not ★ BCNF and 3NF differ only when there are 2+ overlapping candidate keys ★ Every BCNF relation is in 3NF; the converse is NOT always true ★ Test: for every FD A → B, check if A is a candidate key

8.1 When 3NF is NOT BCNF

Classic Example — Course Scheduling:

Table: Schedule(Student, Course, Teacher)

Student	Course	Teacher	Notes
Alice	Math	Prof. Smith	
Alice	Physics	Prof. Jones	
Bob	Math	Prof. Smith	
Carol	Math	Prof. Adams	← *Same course, diff teacher!*

FDs in this table:

- {Student, Course} → Teacher (each student enrolled in a course has one teacher)
- {Student, Teacher} → Course (each student with a teacher is in one course)
- **Teacher → Course (each teacher teaches only one course) ← VIOLATES BCNF!**

Both {Student,Course} and {Student,Teacher} are candidate keys. But Teacher → Course has Teacher on left side which is NOT a candidate key. This is a 3NF table (no transitive dep of non-prime on PK) but NOT BCNF!

8.2 After BCNF Decomposition

Table 1: TeacherCourse(Teacher, Course) — captures Teacher → Course

Teacher (PK)	Course
Prof. Smith	Math
Prof. Jones	Physics
Prof. Adams	Math

Table 2: StudentTeacher(Student, Teacher) — captures enrollment

Student	Teacher
Alice	Prof. Smith
Alice	Prof. Jones
Bob	Prof. Smith
Carol	Prof. Adams

9. Quick Comparison — All Normal Forms

NF	Core Rule	Removes	Requirement	When to Apply
1NF	Atomic values, single-valued attributes	Multi-valued attributes, repeating groups	Primary key exists	Always — baseline for all tables
2NF	No partial dependency	Partial FDs on composite PK	Must be in 1NF	Tables with composite PKs

NF	Core Rule	Removes	Requirement	When to Apply
3NF	No transitive dependency	Non-prime → Non-prime FDs	Must be in 2NF	Standard target for most production DBs
BCNF	Every determinant is a candidate key	Remaining FD anomalies from overlapping CKs	Must be in 3NF	When multiple overlapping candidate keys exist

10. Exercises & Practice Problems

Exercise 1 — Identify Normal Form

Given the following table, identify violations and state which normal form it breaks:

Library(Book_ID, Title, Author, Author_Phone, Genre)

Book_ID	Title	Author	Author_Phone	Genre
B01	Clean Code	Robert Martin	9900001111	Programming
B02	DBMS Concepts	Silberschatz	9900002222	Database
B03	The Pragmatic Programmer	Robert Martin	9900001111	Programming

Questions:

1. Is this table in 1NF? Why or why not?
2. Identify all functional dependencies.
3. Does it violate 3NF? If yes, what is the transitive dependency?
4. Decompose it to 3NF. Write the resulting tables.

Answer:

- 1NF: Yes — all atomic, no repeating groups, Book_ID is PK.
- FDs: Book_ID → Title, Author, Author_Phone, Genre; Author → Author_Phone
- 3NF Violation: Book_ID → Author → Author_Phone is a transitive dependency.
- Decompose to: Book(Book_ID, Title, Author, Genre) and Author(Author, Author_Phone)

Exercise 2

Normalize the following table from UNF to 3NF:

ORDER_TABLE(Order_ID, Customer_ID, Customer_Name, Customer_City, Product_IDs, Product_Names, Quantities)

Additional info: Each order can have multiple products. Customer_ID → Customer_Name, Customer_City. Product_ID → Product_Name.

Step	What to Do	Result Tables
UNF → 1NF	Split multi-valued Product_IDs, Product_Names, Quantities into	OrderItems(Order_ID, Customer_ID, Customer_Name,

Step	What to Do	Result Tables
	rows. Set PK = {Order_ID, Product_ID}.	Customer_City, Product_ID, Product_Name, Qty)
1NF → 2NF	Remove partial deps: Customer_ID → Customer_Name/City (partial — depends only on part of PK). Product_ID → Product_Name (partial).	Order(Order_ID, Customer_ID) | Customer(Customer_ID, Name, City) | Product(Product_ID, Name) | OrderLine(Order_ID, Product_ID, Qty)
2NF → 3NF	Check for transitive deps in each table. No transitive deps remain after 2NF decomposition here.	Same 4 tables — already in 3NF. ✓

Exercise 3

Table: ProjectAssignment(Employee_ID, Project_ID, Department_ID)

FDs given: {Employee_ID, Project_ID} → Department_ID | Department_ID → Project_ID

Question	Analysis	Answer
Is it in 3NF?	No non-prime attribute transitively depends on PK via another non-prime attribute?	YES — it is in 3NF
Is it in BCNF?	Check: Department_ID → Project_ID. Is Department_ID a candidate key?	NO — Dept_ID is not a candidate key. Violates BCNF.
Decompose to BCNF	Separate the violating FD: Dept → Project.	DeptProject(Dept_ID, Project_ID) + EmpDept(Emp_ID, Dept_ID)

Exercise 4 — Spot the Anomaly

For each scenario below, name the anomaly type (Insertion / Update / Deletion):

Scenario	Anomaly Type (Answer)
You want to add a new professor to the system but they haven't been assigned to any course yet, so you can't insert them.	**Insertion Anomaly**
A student drops their only enrolled course, and in doing so, the city of their address is also lost from the DB.	**Deletion Anomaly**
A supplier's address changes, but the same address is stored in 50 rows across the order table — you update only 20.	**Update Anomaly**

Scenario	Anomaly Type (Answer)
A new warehouse is opened but no shipments have been sent from it yet, so it cannot be recorded.	**Insertion Anomaly**

Exercise 5 — Bank Database Normalization

Un-normalized Table: BankData(Account_No, Customer_ID, Customer_Name, Branch_ID, Branch_City, Balance, Loan_IDs, Loan_Amounts)

Rules: One customer can have multiple accounts. One account has one branch. One account can have multiple loans. Branch_ID → Branch_City.

Step	Problem Found	Rule Applied	Tables After
→ 1NF	Loan_IDs and Loan_Amounts are multi-valued	Split into rows, set PK = {Account_No, Loan_ID}	BankData(Acc_No, Cust_ID, Cust_Name, Branch_ID, Branch_City, Balance, Loan_ID, Loan_Amt)
→ 2NF	Cust_ID → Cust_Name: partial dep. Branch_ID → Branch_City: partial dep. Loan_ID → Loan_Amt: partial dep.	Separate each partial dep to its own table	Account(Acc_No,Cust_ID,Branch_ID,Balance) | Customer(Cust_ID,Cust_Name) | Branch(Branch_ID,City) | Loan(Loan_ID,Amt) | AccLoan(Acc_No,Loan_ID)
→ 3NF	No transitive deps remain in any table after 2NF.	All tables already in 3NF ✓	Final: 5 tables — Customer, Account, Branch, Loan, AccLoan

11. Denormalization — When to Break the Rules

Denormalization is the intentional introduction of redundancy into a normalized database to improve read performance. It is used when query speed is more important than storage efficiency.

When to Normalize	When to Denormalize
OLTP systems (frequent inserts/updates)	OLAP / Data Warehouse (heavy reads, reporting)
Data integrity is the top priority	Query performance is the top priority
Storage space is limited	Joins are too expensive across many tables
Frequent updates to many rows	Data is mostly read-only / rarely updated

Normalization is learned by doing, not memorizing!